



Plan De Pruebas

Temporis:

Una herramienta
accesible para el
control y la
optimización de
nuestro tiempo

1. Definición del Alcance de la Calidad

Para garantizar que **Temporis** sea una herramienta fiable y accesible, se han identificado los siguientes módulos y requisitos críticos como objetivos prioritarios del plan de calidad:

1. Definición del Alcance de la Calidad

- Para garantizar que **Temporis** sea una herramienta informática robusta, estable, segura y completamente inclusiva, el alcance de las actividades de control de calidad, aseguramiento técnico e inspección se divide de forma rigurosa en dos grandes bloques organizativos:

1.1. Módulos Críticos del Sistema (Requisitos Funcionales)

- **Gestión de Autenticación y Seguridad Multimodal:** Verificación exhaustiva del acceso seguro, persistencia de sesión y protección de datos de usuario mediante *Firebase Auth*. Esto incluye la validación del registro/*login* tradicional por credenciales, el Social Login con Google, y el flujo de extensión del BiometricPrompt nativo (huella dactilar y reconocimiento facial) para asegurar un control de acceso robusto.
- **Motor de Temporizadores (Core):** Control y monitorización de la exactitud milimétrica en la cuenta atrás, la gestión de hilos en segundo plano mediante corrutinas de Kotlin, y la correcta persistencia de estados del ciclo de vida (activo, pausado, detenido) ante interrupciones externas o llamadas entrantes.
- **Sincronización Cloud Firestore:** Validación de la integridad transaccional de los datos al guardar, actualizar, eliminar y recuperar temporizadores y registros históricos en la nube en tiempo real.
- **Módulo Analítico y Visualización:** Renderizado dinámico y eficiente de la matriz de contribución anual (*heatmap*) mediante la librería *MPAndroidChart*, garantizando que el procesamiento de alta densidad de datos históricos no provoque bloqueos ni ralentizaciones en el hilo principal de la interfaz (*UI Thread*).

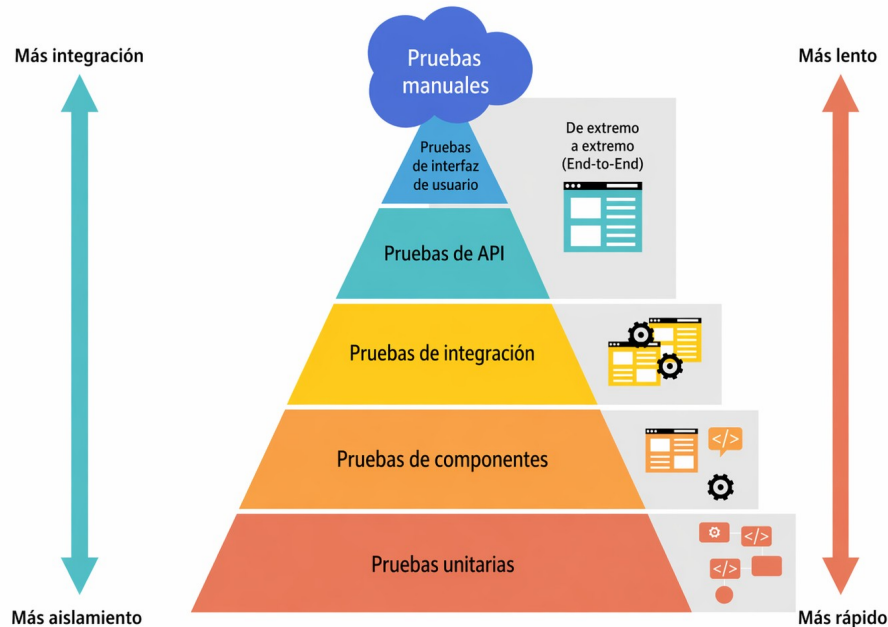
1.2. Requisitos No Funcionales Críticos (Calidad de Software)

- **Accesibilidad Universal (Inclusión Digital):** Cumplimiento estricto de las pautas WCAG aplicadas a entornos móviles. Se audita de forma prioritaria el escalado adaptativo de fuentes (hasta 30sp) sin desbordamiento de componentes, el tamaño mínimo de las zonas de interacción táctil y la perfecta compatibilidad con el modo de alto contraste.
- **Rendimiento y Optimización:** Tiempo de respuesta mínimo en la carga de estadísticas, sincronización eficiente en la nube y consumo optimizado de batería en procesos en segundo plano.
- **Mantenibilidad y Código Limpio:** Control riguroso de la deuda técnica, erradicación de código duplicado, limitación de la complejidad ciclomática y adherencia estricta a la arquitectura limpia MVVM (*Model-View-ViewModel*).

2. Diseño de la Estrategia de Pruebas

Se adopta una estrategia de pruebas multinivel basada en la **Pirámide de Pruebas de Software**, la cual distribuye los esfuerzos de control de calidad desde el aislamiento técnico hasta la validación de la experiencia real del usuario final. Esta estrategia se ve reforzada desde el inicio del ciclo de vida con inspección estática automatizada.

Pirámide de pruebas con pruebas de interfaz de usuario, pruebas de integración y pruebas unitarias



2.1. Tabla Introdutoria de la Estrategia de Pruebas

Tipo de Prueba	Objetivo	Entorno y Herramientas	Responsable
Unitarias	Validar la lógica aislada de las clases (<i>User</i> , <i>Timer</i> , <i>TimeRegister</i>).	Android Studio / JUnit & Mockito	Desarrollador
Integración	Verificar la comunicación correcta entre la <i>App</i> y <i>Firebase</i> (CRUD).	Emulador Android / <i>Firebase Emulator</i>	Desarrollador
Sistema (UI)	Simular el flujo completo del usuario: <i>Login</i> -> Crear <i>Timer</i> -> Ejecutar.	Dispositivo Físico / <i>Espresso Framework</i>	QA / Desarrollador (Se asumirá el rol de <i>QA Tester</i> , aplicando una separación de mentalidad entre las fases (desarrollo y <i>testing</i>), preservando la objetividad.)
Aceptación	Validar que el diseño final coincide con los prototipos definidos una de las tarea.	<i>Beta Test</i> (Usuario final)	Usuario / Analista
Seguridad	Comprobar que las <i>Security Rules</i> de <i>Firestore</i> impiden acceso no autorizado.	Consola de <i>Firebase</i> / Pruebas de inyección	Desarrollador

2.2. Tabla Detallada de Estrategia de Pruebas

Tipo de Prueba	Objetivo Crítico	Entorno y Herramientas	Responsable	Logros, Retos y Avances Recientes
Inspección Estática	Análisis automatizado de la calidad del código fuente. Detección temprana de <i>bugs</i> , <i>code smells</i> , deuda técnica y vulneración de estándares.	SonarQube Local Server / Gradle Plugins	Desarrollador / QA Tester	Logro: Integración exitosa de la pasarela de calidad (<i>Quality Gate</i>). Se refactorizó el <i>ViewModel</i> de los temporizadores reduciendo la complejidad ciclomática y se limpiaron variables duplicadas en las corrutinas de Kotlin, alcanzando un índice de deuda técnica "A".
Unitarias	Validar de forma aislada e independiente las funciones matemáticas de conversión de tiempo y la lógica pura de negocio en los <i>ViewModels</i> . Clases objetivo: <i>User</i> , <i>Timer</i> , <i>TimeRegister</i> .	Android Studio / JUnit 4 & MockK (o Mockito)	Desarrollador	Avance: Monitorización de la cobertura de código (<i>Code Coverage</i>) de las pruebas de regresión centralizada directamente en el panel de <i>SonarQube</i> , con la intención de asegurar un mínimo del 85% de cobertura en clases y controladores.
Integración	Verificar la correcta comunicación, serialización e intercambio de datos entre los repositorios locales de la <i>App</i> y los <i>endpoints cloud</i> de <i>Firebase</i> (operaciones <i>CRUD</i>).	Emulador Android / <i>Firebase Local Emulator Suite</i>	Desarrollador / QA Tester	Reto Superado: Controlar adecuadamente las excepciones de red y asegurar el mapeo exacto de los documentos de <i>Firestore</i> en situaciones de conectividad intermitente o Modo <i>Offline</i> .
Sistema (UI)	Simular flujos completos de interacción dentro de las pantallas prototipadas, navegabilidad por el <i>Bottom Navigation Bar</i> y robustez del hilo de ejecución al pulsar el botón "Timer -> Ejecutar".	Dispositivo Físico / <i>Espresso Framework</i> / Emulador Android	"QA Tester"	Avance: Automatización del flujo de <i>Login</i> y del 'modal' de creación de tareas mediante <i>scripts</i> de <i>Espresso</i> , mitigando fallos por referencias nulas (<i>NullPointerException</i>).

Tipo de Prueba	Objetivo Crítico	Entorno y Herramientas	Responsable	Logros, Retos y Avances Recientes
Accesibilidad	Garantizar que todos los componentes visuales cumplan con un área táctil mínima de 48x48dp y que las etiquetas sean interpretadas adecuadamente por herramientas de asistencia. Verificación de contrastes.	<i>Accessibility Scanner</i> (Google) / <i>TalkBack</i> / Lector de pantalla	Usuario Final / Analista de Accesibilidad	Hito: Validación del comportamiento del slider dinámico de fuentes. Se constató que TalkBack verbaliza de manera correcta e inclusiva los estados de los temporizadores circulares activos. El diseño coincide con lo validado en la tarea realizada.
Seguridad	Comprobar que las reglas de escritura y lectura en la base de datos cloud estén completamente blindadas por token de usuario.	Consola de <i>Firebase</i> / Pruebas de Inyección de <i>Tokens</i>	Desarrollador	Hito: Verificación de las <i>Security Rules</i> de <i>Firestore</i> , garantizando criptográficamente que un usuario autenticado no pueda leer ni manipular las colecciones ni documentos pertenecientes a otro usuario.

3. Secuencia de Ejecución Lógica

La ejecución de las pruebas en **Temporis** no se plantea de manera aleatoria, sino que obedece a una estricta secuencia de dependencia técnica y madurez del código para optimizar costes y tiempos de corrección:

- Fase 1: Verificación Estática Automática (SonarQube):** Es el primer filtro de calidad e inspección. Cada cambio significativo en el código fuente es escaneado localmente. Si el código no supera los umbrales fijados en el *Quality Gate* (por presencia de *code smells* o vulnerabilidades de seguridad), se detiene el flujo de integración hasta que sea subsanado de raíz.
- Fase 2: Verificación de Código Puro (Unitarias):** Se ejecutan los test unitarios con JUnit para certificar que las modificaciones no han corrompido las reglas operacionales del motor de tiempo ni los cálculos base del estado del temporizador. Es la base de la estabilidad del sistema.
- Fase 3: Conectividad y Persistencia (Integración):** Una vez que la lógica interna es sólida, se levanta el entorno emulado de Firebase para validar que los datos transaccionales fluyen y se sincronizan de forma idónea en la nube sin errores de red o formato.
- Fase 4: Flujos de Usuario (Sistema/UI):** Mediante Espresso, se comprueba la interfaz de usuario en su totalidad, simulando los recorridos del usuario real a lo largo de las vistas de la aplicación para asegurar transiciones limpias y consistentes (especialmente en el modo oscuro).
- Fase 5: Validación de Accesibilidad y Cierre (Aceptación):** Pruebas de campo finales con las herramientas nativas de diagnóstico de Google (TalkBack) y revisiones de diseño para confirmar que el producto final cumple de forma inclusiva con los objetivos y el alcance definidos en la planificación de la tarea “Tr_09” (Apartado: Planificación del Ciclo de Vida (Gantt)).